

WIRELESS SENSOR NETWORKS,

LABORATORY GUIDE 1

Design, Implementation and Calibration of a Multi-Sensor Edge Node

Submission Date	30 th sept 2026
Submission	Repository + short technical report + Simulation + live demonstration

1. Laboratory Overview

In this laboratory you will build a small, reliable sensor node that can acquire measurements from multiple sensors, and validate those measurements, perform basic processing locally, make a simple edge decision, and provide machine-readable data for analysis in Python. The measurements you collect in this lab will later become inputs to IoT analytics and machine-learning models.

2. Competency and Learning Outcomes

By the end of the laboratory, you should be able to:

1. Configure and program an ESP32-class microcontroller.
2. Interface at least three sensors using appropriate digital, analog or I²C interfaces.
3. Acquire, timestamp and validate sensor measurements.
4. Apply basic filtering and feature calculations locally on the edge device.
5. Generate structured sensor data for automated analysis in Python.
6. Characterize sensor response, variability and basic uncertainty.
7. Document a reproducible sensor-node design.

3. Scenario

You are an engineering team developing a low-cost sensing node for a future IoT application such as smart agriculture, environmental monitoring, energy management or smart buildings. The node must be capable of reliable measurement in a resource-constrained environment. You are required to :

1. Acquire measurements from multiple sensors.
2. Detect invalid or implausible values.
3. Calibrate at least one measurement channel.
4. Perform basic filtering or feature extraction locally.
5. Generate a timestamped, machine-readable data stream.
6. Produce a simple local decision or status indicator.

7. Provide data that can be analyzed and visualized using Python.

4. Equipment and Software

4.1 Hardware (From the lab)

1. ESP32 and ESP32-S3 sensorkit.
2. Breadboard and jumper wires.
3. One temperature sensor or equivalent.
4. One analog light sensor: LDR with resistor divider or equivalent.
5. One digital motion sensor: PIR.
6. Optional: MPU6050/MPU9250 IMU, OLED display, buzzer or LED.
7. Reference multimeter or other available reference instrument for calibration.
8. USB cable and computer.

4.2 Software

1. Arduino IDE or ESP-IDF.
2. Python 3, Jupyter Notebook, NumPy, Pandas, Matplotlib and SciPy, Git/GitHub
3. Wokwi for simulation and troubleshooting.

5. Safety and Good Laboratory Practice

1. Power the ESP32 and sensors from the specified voltage; do not connect a 5 V signal to a 3.3 V-only GPIO input.
2. Switch off power before changing wiring.
3. Do not short the 3.3 V or 5 V rails.
4. Keep water and conductive materials away from powered electronics.
5. Use a reference instrument carefully and consistently during calibration.
6. Record unexpected behavior rather than silently deleting measurements.

6. Build and Acquire

1. Create a new firmware project and upload a simple test program. Confirm that the serial monitor displays a startup message.
2. Build the circuit using at least three sensing modalities.
3. Acquire Raw Sensor Data. Implement a periodic sampling loop. Start with a 1 Hz sampling interval unless the sensor requires a different rate.

```
timestamp,temp,humidity,light,motion
1712,25.4,67.8,0.72,0
1713,25.5,67.7,0.74,1
```

- Record at least 5 minutes of baseline measurements under stable laboratory conditions.

7.1 Sensor Characterization

Change the environment deliberately and observe how the sensor responds.

Sensor	Controlled change	What to measure
Temperature/Humidity	Move between stable conditions or compare different locations	Response, variability, bias
LDR	Dark → normal room → bright light	Range, repeatability, response
PIR	No motion → person enters field of view	Detection behavior, false triggers
IMU (optional)	Static → movement	Signal range, noise, variability

7.2: Local Edge Processing

Implement at local processing operations on the ESP32.

- Five-sample moving average for temperature or another continuous measurement.
- Rate of change, $\Delta x = x(t) - x(t-1)$.
- Threshold/event detection.
- A simple feature such as mean, RMS or standard deviation for an IMU signal.

Example:

```
if calibrated_temperature > 30.0:
    status = "HIGH_TEMP"
else:
    status = "NORMAL"
```

7.3: Python Data Acquisition and Analysis

Use Python to collect the machine-readable serial stream, containing at least 600 rows of sensor readings and export CSV file . Create a Pandas DataFrame and calculate descriptive statistics.

- Count and duration of observations.
- Mean, minimum and maximum.
- Standard deviation.
- Number of invalid measurements.
- Number of detected events.

7.4: Visualization

Create at least four plots:

- Temperature vs time.

- Humidity vs time.
- Raw vs filtered sensor values.
- An optional fifth plot may show motion events or another event-driven variable.

8.1 Engineering Analysis Questions

1. What are the major subsystems of your sensor node and what is the role of each?
2. What is the effective sampling rate of each sensor?
3. Which sensor showed the greatest variability and why?
4. What systematic error did calibration reveal?
5. Did calibration reduce MAE and RMSE? By how much?
6. What effect did filtering have on noise and signal responsiveness?
7. Which information should be processed locally at the edge, and which information should be transmitted?
8. What would happen to energy consumption if every raw measurement were transmitted?

8.2. Advanced Challenge

- a) From an IMU and calculate acceleration magnitude, mean and standard deviation.
- b) Implement a simple local data buffer so that short communication outages do not immediately lose data.
- c) Publish the structured data using MQTT and inspect the messages with a desktop MQTT client.
- d) Estimate the effect of changing the sampling interval from 1s to 5s on data volume and expected energy use.

9.1 Required Deliverables

- a) Wokwi simulation.
- b) Working ESP32 sensor-node prototype.
- c) Circuit/wiring diagram.
- d) Firmware source code (hosted on github)
- e) Python/Jupyter analysis notebook.
- f) Raw and processed datasets.
- g) Calibration data and fitted calibration model.
- h) At least four plots.
- i) README with setup and reproduction instructions.
- j) Technical report using the structure below.

9.2. Technical Report Structure

- a) Title and team members
- b) Objective and competency being demonstrated
- c) System architecture and hardware
- d) Sensor interfaces and sampling design
- e) Sensor characterization and data quality
- f) Edge processing and local decision logic
- g) Python data analysis and visualization
- h) Discussion and engineering interpretation
- i) Limitations and recommended improvements
- j) Conclusion
- k) References (IEEE format)
- l) Repository link / file structure

9.3 Recommended Git hub Repository Structure.

```
lab1/
├── firmware/
│   └── sensor_node/
├── python/
│   ├── acquisition.ipynb
│   └── analysis.ipynb
├── data/
│   ├── raw/
│   └── processed/
├── figures/
├── docs/
│   ├── circuit_diagram.png
│   └── lab_report.pdf
└── README.md
```